

PROYECTO DE CURSO

750021C - DESARROLLO DE SOFTWARE II



FACULTAD DE INGENIERÍA
Escuela de Ingeniería de Sistemas y
Computación

FEBRERO - JUNIO / 2026

UniTicket: Gestión de Incidencias Técnicas

1. Introducción

En el marco de la asignatura Desarrollo de Software II, surge UniTicket como un proyecto integrador diseñado para enfrentar los retos reales de la ingeniería de software moderna. Este sistema no es solo una aplicación de gestión de incidencias técnicas; es el laboratorio donde se aplicará la filosofía DevOps, la automatización de procesos y el despliegue en entornos de nube.

Por medio de este proyecto de clase, se implementará un sistema de información web para el reporte y gestión de fallas técnicas en los laboratorios de una universidad de la región.

El énfasis principal es la implementación de un ecosistema DevOps profesional utilizando un servidor Jenkins centralizado en la nube para automatizar la Integración Continua (CI) y el Despliegue Continuo (CD).

2. Requerimientos Técnicos

- **Backend de la Aplicación:** Desarrollo en Python (preferiblemente Django) o cualquier otro Framework o CMS web.
- **Base de Datos:** Motor relacional (PostgreSQL o MySQL) en contenedor (Docker o Podman).
- **Infraestructura:** Cada equipo recibirá una instancia de Jenkins administrable en el Servidor Privado Virtual (VPS) del curso.
- **Control de versiones:** Repositorio en GitHub con flujo Gitflow estricto.
- **Automatización:** Pipeline definido en un archivo **Jenkinsfile** dentro del repositorio.

3. Requerimientos Funcionales de la Aplicación (UniTicket)

Para el desarrollo del producto tecnológico, el sistema UniTicket deberá implementar las siguientes funcionalidades mínimas, simulando el flujo básico de gestión de incidencias en los laboratorios:

3.1. Gestión de Acceso y Roles (Autenticación)

El sistema debe permitir el inicio de sesión para al menos dos tipos de roles: **Usuario Estándar** (estudiantes/docentes que reportan la falla) y **Técnico/Administrador** (personal encargado de la solución).

3.2. Módulo de Reporte de Incidencias (Creación de Tickets)

Los usuarios estándar deben poder crear un nuevo ticket indicando:

- Ubicación (ej. Laboratorio de Redes, Sala de Software).
- Identificador del equipo (ej. PC-05).
- Descripción detallada del problema.
- Fecha de reporte (automática).

3.3. Módulo de Reporte de Incidencias (Creación de Tickets)

Tablero de Control (Dashboard)

- El sistema debe proveer una vista principal (lista o tabla) donde se visualicen los tickets registrados.
- Cada ticket debe mostrar un estado visual claro: **Abierto**, **En Progreso** o **Resuelto**.

3.4. Módulo de Reporte de Incidencias (Creación de Tickets)

Gestión y Actualización de Tickets (Solo Técnicos)

- Los usuarios con rol de Técnico deben poder acceder al detalle de un ticket, cambiar su estado (ej. pasar de Abierto a Resuelto) y agregar un comentario o nota de resolución.

4. Objetivos del Proyecto

- **Implementar una cultura DevOps:** Aplicar las diferentes prácticas de DevOps (Integración, Entrega y Despliegue Continuo), reconociendo sus beneficios y retos para agilizar el desarrollo de software.
- **Automatización y Calidad:** Asegurar la automatización y el monitoreo de todas las fases de construcción del software mediante pruebas automatizadas.
- **Configuración de Entornos:** Configurar escenarios reales de CI/CD para un proyecto particular, integrando herramientas usadas en la industria.
- **Evaluación de Calidad:** Valorar factores de calidad estandarizados como la seguridad, eficiencia y portabilidad en el producto de software desarrollado.

5. Estructura de la Infraestructura Asignada

Cada equipo de estudiantes recibirá un "Paquete de Infraestructura" que incluye:

- **Instancia de Jenkins:** Accesible vía <http://45.55.145.98:8080>.
- **Credenciales Administrativas:** Usuario y contraseña para gestionar plugins, credenciales de GitHub y Jobs.
- **URL de Webhook:** Dirección única para configurar en GitHub: <http://45.55.145.98:9000/github-webhook/>
- **Puertos de Aplicación:** Rango de puertos asignados para que su contenedor de UniTicket sea accesible desde internet (ej. [7000](#)).

6. Metodología de Control de Versiones

Para asegurar un desarrollo organizado de la aplicación y facilitar la Integración Continua (CI), es obligatorio el uso de un flujo de trabajo (Git Flow, GitHub Flow, GitLab Flow, etc.) ya que este esquema permite separar el código en desarrollo de las versiones estables.

Estructura de Ramas Mínima

- **main (Producción):** Contiene el código estrictamente estable y desplegable. Cada cambio aquí debe provenir de un merge exitoso desde el branch **develop**.
- **develop (Integración):** Es la rama principal de trabajo del equipo. Aquí se integran todas las funcionalidades antes de pasar a producción.
- **feature/nombre-tarea:** Ramas temporales creadas para desarrollar una funcionalidad específica del sistema UniTicket. Al terminar, se debe solicitar un Pull Request hacia **develop**.
- **hotfix/nombre-tarea:** Ramas urgentes para corregir errores en la versión de producción. (En caso de usarse debe justificarse por qué en el informe final)

Reglas de Operación y Calidad

- **Pull Requests (PR):** No se permite la integración directa de código a **develop** o **main**. Todo cambio debe pasar por un PR.
- **Code Review:** Cada PR debe ser revisado y aprobado por al menos un compañero del equipo antes de ser mezclado. En caso de tener comentarios o requerir aclaraciones del cambio objeto de Code Review, se evaluará la calidad del feedback proporcionado.
- **Validación por Jenkins:** El servidor de Jenkins debe estar configurado para ejecutar el *pipeline* ante la creación o actualización de un PR. Si las pruebas o el análisis estático fallan, el código no podrá ser integrado.

7. Resultados de Aprendizaje

- **R.A.1 (Identificación):** Reconocer los beneficios de DevOps al gestionar un servidor real que automatiza procesos de construcción.
- **R.A.2 (Configuración):** Operar y mantener un escenario de CI/CD profesional, gestionando fallos en el pipeline y asegurando la estabilidad del código.
- **R.A.3 (Comunicación):** Elaborar informes técnicos que justifiquen la arquitectura de infraestructura seleccionada.
- **R.A.4 (Colaboración):** Participar activamente en las reuniones periódicas y realizar revisiones de código responsables mediante Pull Requests.

8. Entregables

8.1. Ecosistema de Automatización (Pipeline CI/CD)

En este entregable se valida la configuración de escenarios de Integración y Despliegue Continuo. Incluye:

- **Script de Pipeline:** Un archivo **Jenkinsfile** que define todas las etapas del ciclo de vida del software: *Checkout, Linting, Testing, Build y Deploy*.
- **Configuración de Webhooks:** Evidencia de la comunicación en tiempo real entre el repositorio y el servidor de automatización para disparar ejecuciones ante cada *Pull Request*.
- **Reportes de Calidad Automatizados:** Resultados de las pruebas unitarias y del análisis estático de código (SAST) integrados directamente en el flujo de trabajo.

8.2. Producto Tecnológico (UniTicket)

En este entregable es donde se evidencia la capacidad de desarrollo y despliegue e incluye:

- **Repositorio en GitHub:** Debe ser público y seguir estrictamente la metodología Gitflow con ramas **main**, **develop**, **feature/xxx** y **hotfix/xxx**. Se evaluará el historial de *Pull Requests* y los *Code Reviews* realizados entre compañeros.
- **Aplicación Dockerizada:** El sistema debe estar compuesto por al menos dos contenedores uno para la aplicación y uno para la base de datos, dichos contenedores estarán orquestados mediante un archivo **docker-compose.yml**.
- **Instancia en Producción:** La aplicación debe ser accesible desde internet a través de la IP del VPS y el puerto asignado a cada grupo (ej. **http://45.55.145.98:700x**).

8.3. Documentación Técnica e Informes

Este documento escrito recopila las justificaciones técnicas de cada decisión tomada en la configuración del ecosistema de automatización, evidencia la capacidad de comunicación técnica e incluye como mínimo los siguientes ítems:

- **Especificación del Ecosistema CI/CD**
 - **Diagrama de Flujo del Pipeline:** Representación visual de cada etapa (*stages*) desde el **git push** hasta el despliegue final en el VPS.
 - **Configuración de Webhooks:** Captura y explicación de cómo el repositorio de GitHub se comunica automáticamente con el servidor Jenkins.

- **Gestión de Entornos y Contenedores**
 - **Diagrama de Infraestructura:** Representación visual de la conexión entre el repositorio de GitHub, el servidor Jenkins y el entorno de despliegue en el VPS.
 - **Orquestación con Docker Compose:** Descripción de cómo se gestiona la red interna entre el contenedor de la aplicación UniTicket y el de la base de datos.
 - **Diagrama de Red:** Topología de la red de contenedores. Explicación de cómo se aíslan los contenedores y cómo se comunican la aplicación y la base de datos a través de redes virtuales.
- **Reporte de Aseguramiento de la Calidad**
 - **Métricas de Pruebas Unitarias:** Resumen del porcentaje de cobertura de código (Code Coverage) y resultados de la ejecución automatizada en Jenkins.
 - **Resultados de Linting y SAST:** Informe de los errores detectados por el análisis estático de código y cómo fueron corregidos para cumplir con los estándares de calidad.
- **Conclusiones y Lecciones aprendidas**
 - Registro de los retos técnicos encontrados durante la ejecución del proyecto y cómo fueron superados colaborativamente.
 - **Análisis de Fallos:** Descripción de incidentes donde el pipeline falló y la importancia de que el sistema detuviera el despliegue erróneo.